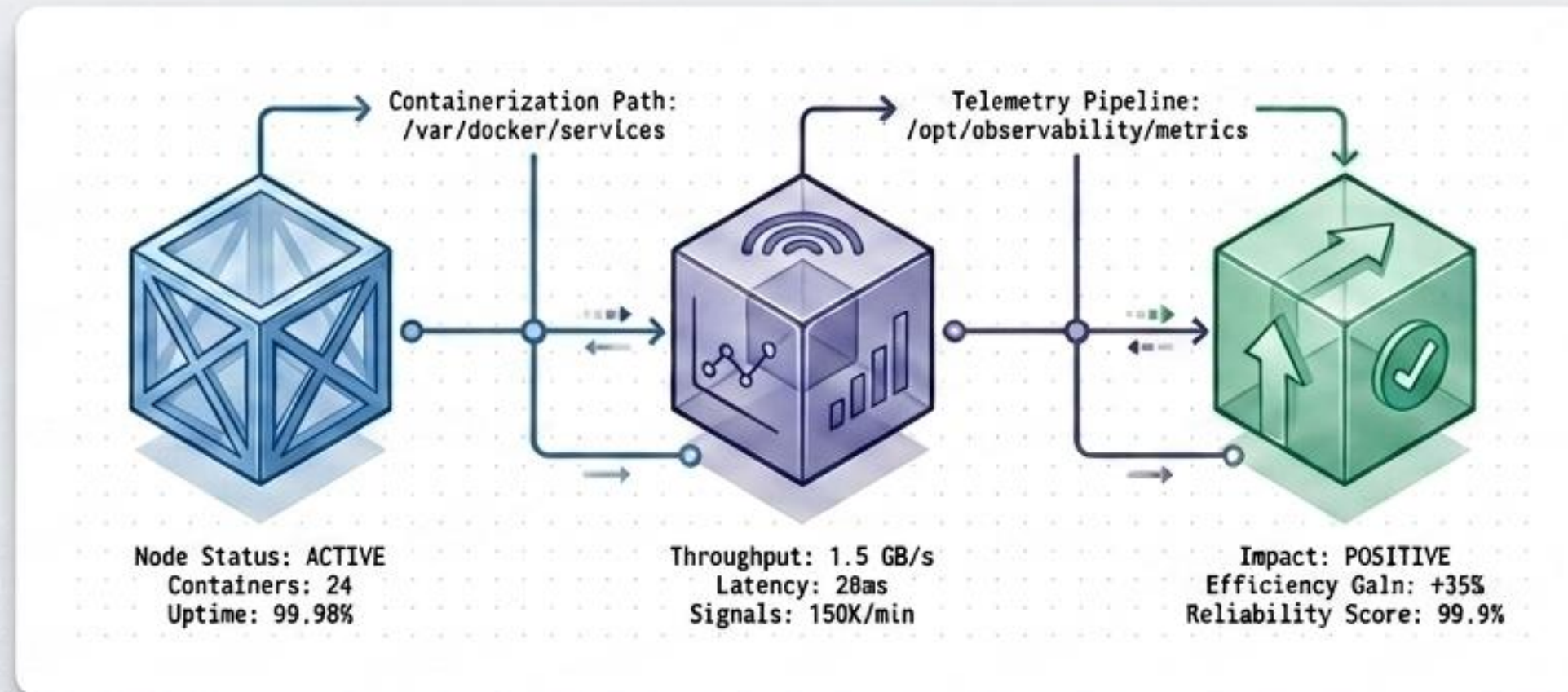


Alentapp: PREPARANDO PARA PRODUCCIÓN

Despliegue y observabilidad

Grupo 14 | Ingeniería y Calidad de Software 2026



Integrantes: Egüen Agustina, Montes Joaquin, Pascucci Agostina, Perez Nicolas, Smith Justina, Talavera Santiago

Auditoría Inicial: Un Entorno Vulnerable



Credenciales en Texto Plano

password123 expuesta en `docker-compose.yml`.



Imágenes Monolíticas

Ausencia de multi-stage builds. Código fuente y devDependencies expuestos.



Ejecución como Root

Proceso `Node.js` operando con privilegios máximos.



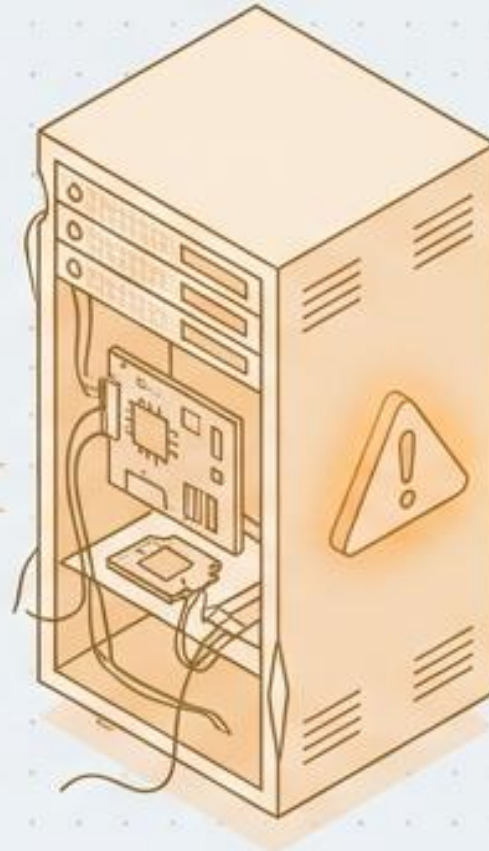
Montaje de Directorio Raíz

`bind-mount (./app)` expone el workspace completo al contenedor.



Sin Límites de Recursos

Riesgo de caída del host completo por consumo descontrolado de CPU/Memoria.



Optimización Estructural: El Embudo Multi-Stage

Stage 1: deps

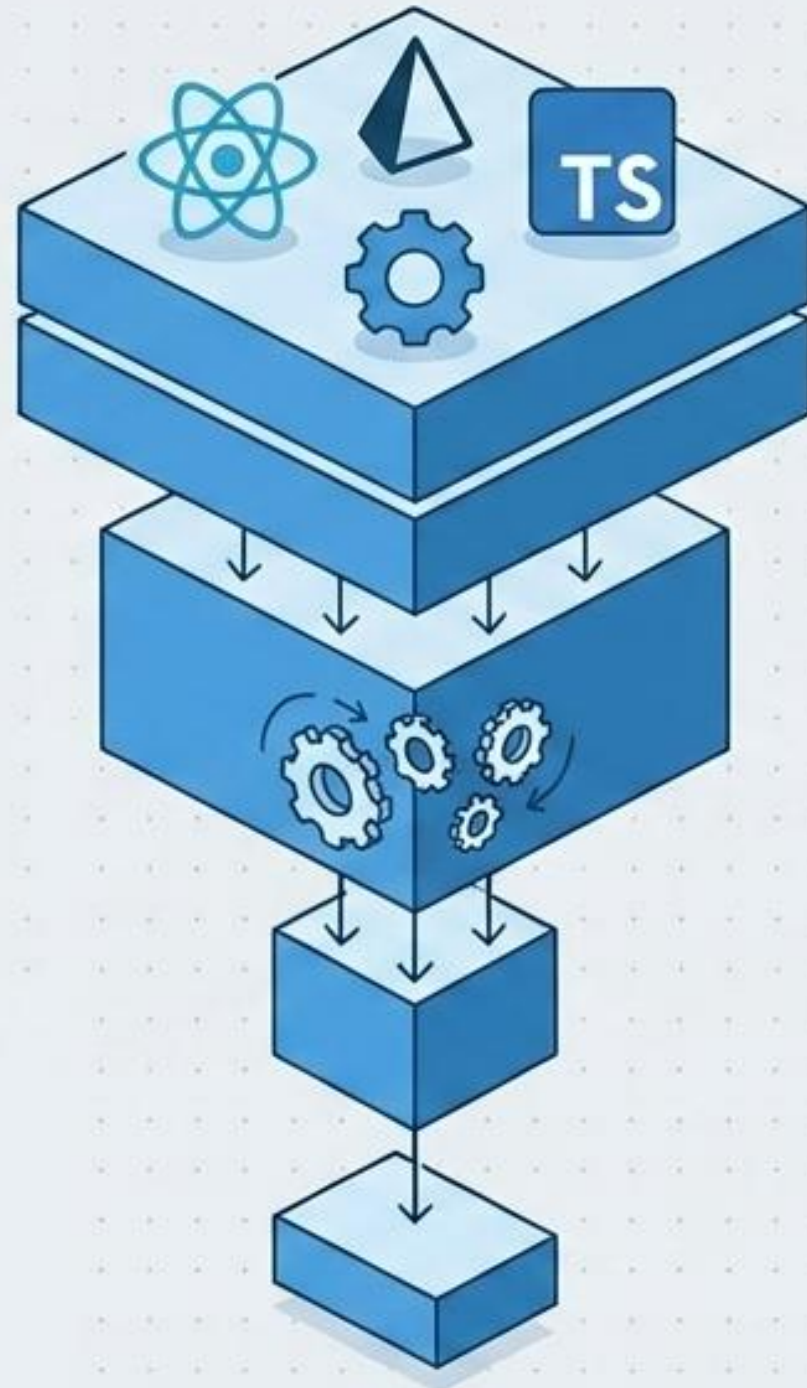
Instalación completa.
Retiene peso innecesario.

Stage 2: build

Compilación de código
fuente (TS a JS, Vite build).

Stage 3: runtime

Solo JS compilado,
dependencias de producción
y Nginx (para Web).



Impacto Web:
Reducción del **88.8%**

De **857 MB** a **93.7 MB**

Anatomía de un Contenedor Blindado



read_only: true

El sistema de archivos es inmutable. Bloquea modificaciones en caliente.



cap_drop: ALL

Privilegios a nivel de kernel removidos, conservando únicamente NET_BIND_SERVICE.



Usuario node

Ejecución estricta non-root, eliminando vectores de escalamiento de privilegios.

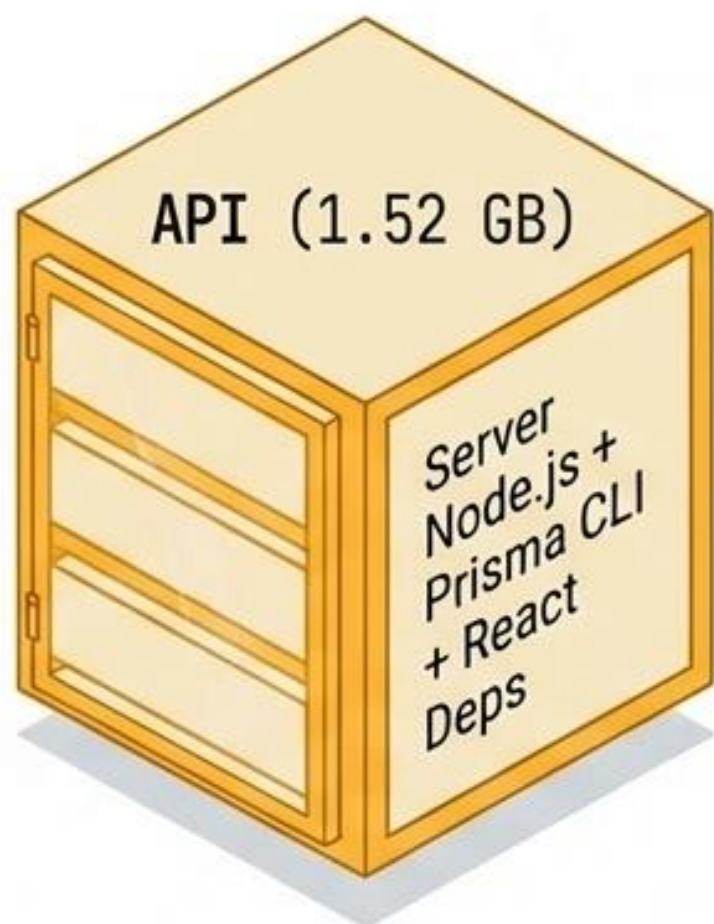


.env.prod

Inyección dinámica de secretos. Cero credenciales compiladas en la imagen.

El Desafío de Prisma: Desacoplando las Migraciones

Antes (Monolítico)



El CLI de Prisma forzaba la inclusión de herramientas pesadas en la imagen de ejecución.

Tamaño final de API:
434 MB (-71.4%)

Después (Patrón Sidecar)

api-migrate (Sidecar)
Ejecuta migraciones de DB y termina.



api (Runtime)
Sirve tráfico HTTP. Espera a api-migrate (condition: **service_completed_successfully**).

Modelo Conceptual de Telemetría (Método RED)

http.requests.total



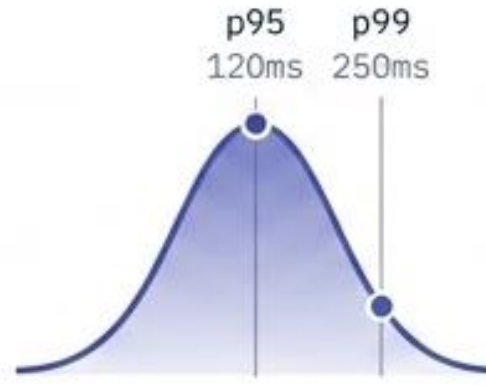
Peticiones por segundo

http.requests.errors



Tasa de error 4xx/5xx

http.request.duration



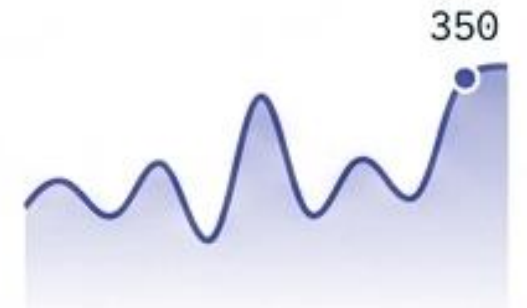
Latencia p95/p99

process.memory.usage



Consumo de RAM

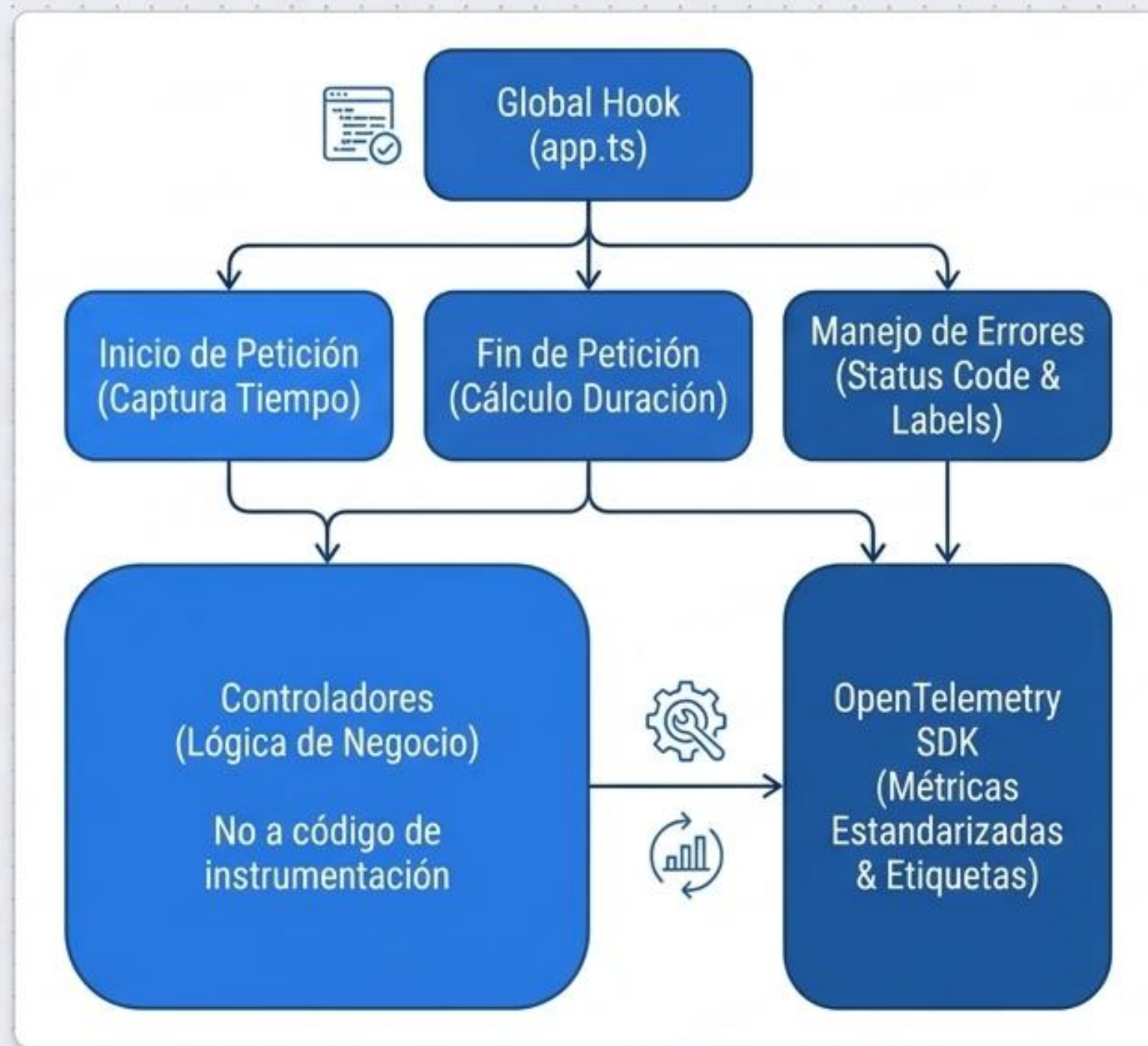
http.requests.active



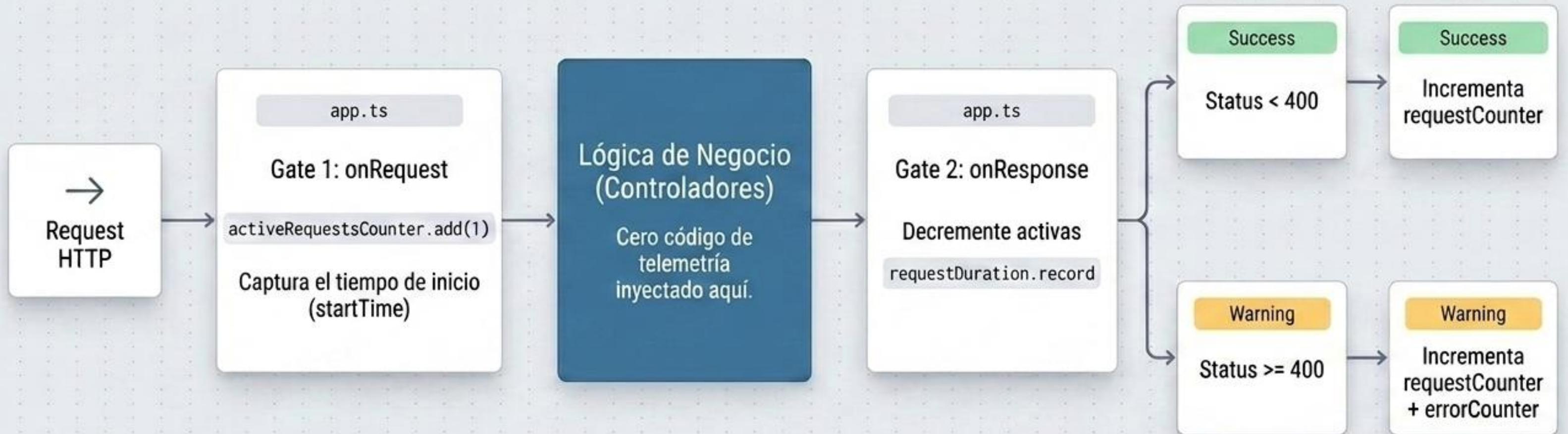
Peticiones concurrentes

Estrategia de Instrumentación: El Pivote a Hooks Globales

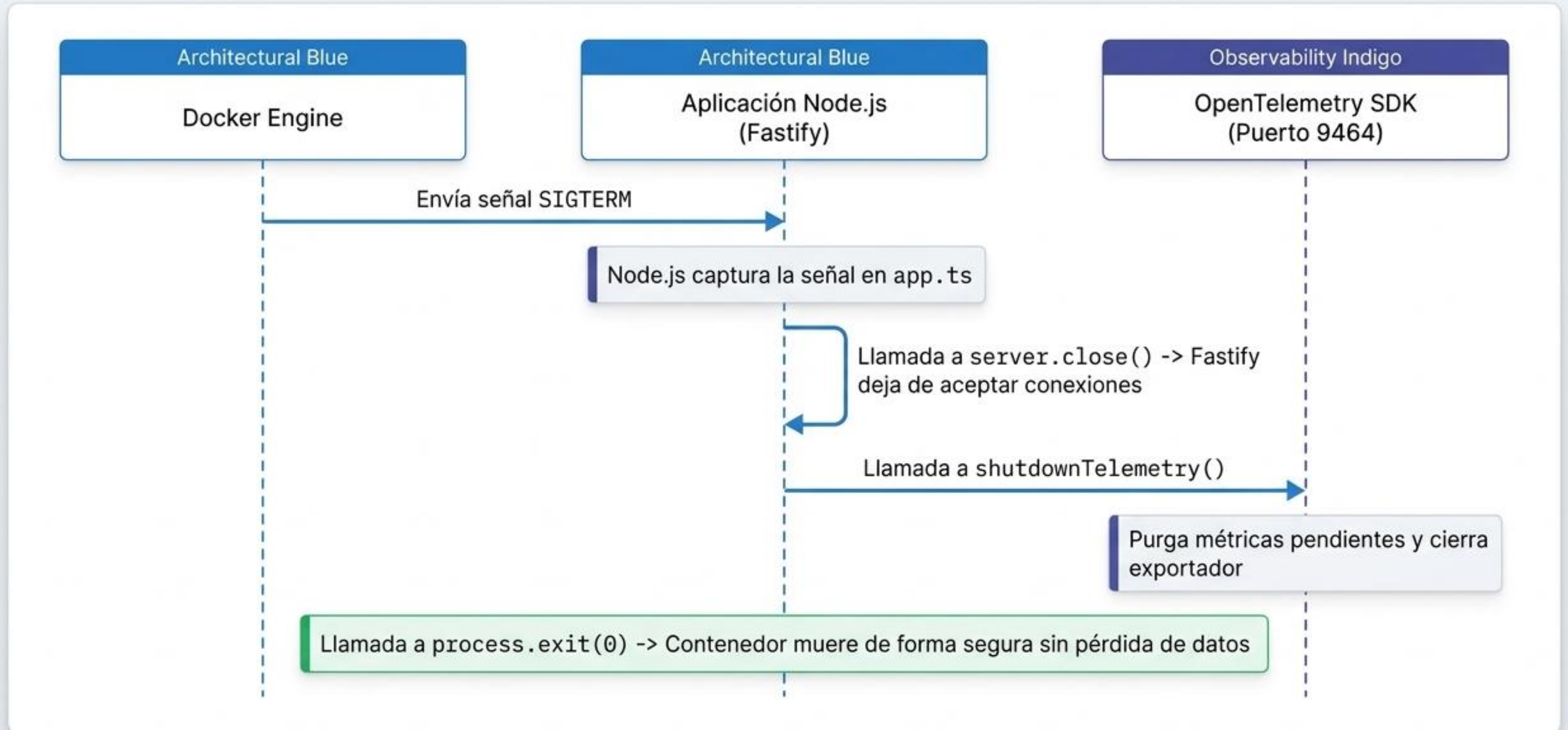
Dimensión	Manual (Por Controlador)	Hooks (Global en app.ts)
Cobertura	Requiere código en cada ruta.	100% automático.
Clean Code	Ensucia la lógica de negocio con try/finally.	Separación estricta de responsabilidades.
Consistencia	Riesgo alto de inconsistencia en labels.	Labels estandarizados (method, route, status).
Manejo de Errores	Dificultad para calcular Tasa de Error real.	Diferenciación precisa entre peticiones exitosas y errores.



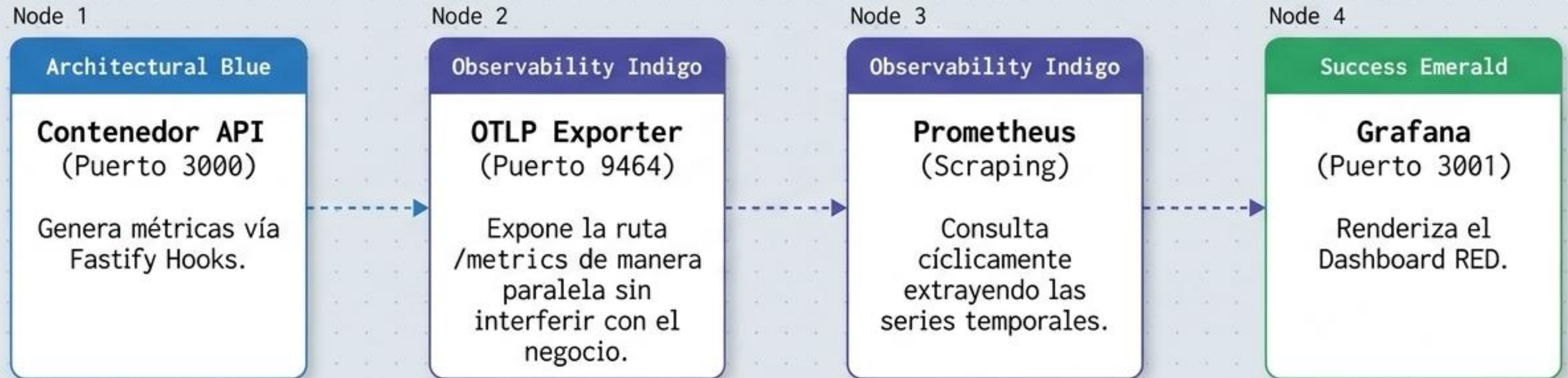
El Flujo del Interceptor de Observabilidad



Gestión del Ciclo de Vida: Apagado Seguro (Graceful Shutdown)



Arquitectura de Observabilidad End-to-End



El aislamiento de puertos garantiza que el monitoreo no afecte la performance del servicio principal.

Impacto Final: Matriz de Rendimiento Dev vs. Prod

Métrica	Antes (desarrollo)	Después (producción)	Mejora
Tamaño imagen API	1.52 GB	434 MB	~71.4% menos (reducción de ~1.1 GB)
Tamaño imagen Web	857 MB	93.7 MB	~88.8% menos (reducción de ~763 MB)
Tiempo de startup API	2m45.357s (time docker compose up -d api: DB healthcheck 36.6s + API 35.1s)	0m23.925s (time docker compose -f docker-compose.prod.yml up -d api: DB healthcheck 6.8s + API migrate 16.4s + API 0.3s)	~85.5% más rápido en iniciar (stack completo)
Memoria API (startup)	95.57 MiB	28.56 MiB	~70% menos en pico de arranque
Memoria API (startup)	95.57 MiB	28.56 MiB	~70% menos en pico de arranque
Endpoints accesibles	curl localhost:3000/...	curl localhost:3000/... y curl localhost:9464/metrics	Instrumentación activa (métricas de telemetría y salud disponibles)
Frontend vía nginx	— (servidor dev Vite, puerto 5173)	curl localhost/ (puerto 80 nativo)	Servidor Nginx integrado para archivos estáticos óptimos

La reestructuración de contenedores y la instrumentación estratégica resultaron en un sistema altamente resiliente, observable y ultraligero.

Lecciones Aprendidas y Próximos Pasos

Optimización Arquitectónica

- Evitar acoplamiento: De try/catch manuales a Hooks globales.
- Coordinación en equipo: Ramas por dominio y pair programming para evitar conflictos.

Observabilidad Integral

- Seguridad proactiva: Sistema de archivos read-only y usuarios no-root.
- Vendor-Agnostic: Instrumentación con OpenTelemetry para independencia futura.

Resultados Tangibles

- Race conditions: Scripts de inicialización (entrypoint.sh) para dependencias.
- Mentalidad Prod vs Dev: Imágenes multi-stage ultraligeras.